

Experiment No. 01

Build and Test Your First Machine Learning Classifier using K-Nearest Neighbors (KNN)

Aim:

To build and evaluate a K-Nearest Neighbors (KNN) classifier using the Iris dataset, analyze the effect of different values of **k**, and understand how distance-based classification works in vector space.

Objective:

After completing this experiment, students will be able to:

1. Load and explore the Iris dataset.
2. Understand the dataset structure, feature names, and class distribution.
3. Visualize relationships between features using Seaborn Pairplots.
4. Train a K-Nearest Neighbors classifier.
5. Compare classifier performance for different values of **k**.
6. Understand Euclidean distance and feature vector representation.
7. Interpret class probabilities using `predict_proba()`.
8. Document results for reproducibility.

Pseudo Algorithm :

Input: Iris dataset, Value of K (1, 3, 5, 7)

Output: Trained KNN model, Classification accuracy, Class probability predictions

Steps:

1. Start.
2. Import required libraries (numpy, pandas, matplotlib, seaborn, sklearn).
3. Load the Iris dataset.
4. Display dataset information:
 - o Shape
 - o Feature names
 - o Target classes
5. Visualize:
 - o Species distribution
 - o Pairplot of all features.
6. Split dataset into:
 - o Training set (70%)
 - o Testing set (30%)
7. For each value of **K = 1, 3, 5, 7**
 - o Create a KNN classifier.
 - o Train the model using training data.

- Predict test labels.
 - Calculate training accuracy.
 - Calculate testing accuracy.
 - Obtain class probability using predict_proba().
8. Compare accuracies for different values of K.
 9. Interpret the effect of K on bias and variance.
 10. (Optional) Plot decision boundaries.
 11. Save notebook and document observations.
 12. Stop.

Sample Outcome:

Sample Accuracy Table		
K Value	Training Accuracy	Testing Accuracy
1	100%	95.6%
3	98.1%	97.8%
5	97.1%	97.8%
7	96.2%	95.6%

(Values may vary slightly depending on random train-test split.)

Sample Observation:

- Pairplots show that Petal Length and Petal Width provide the best separation between Iris species.
- Smaller K values fit the training data more closely and may overfit.
- Larger K values provide smoother decision boundaries but may underfit.
- KNN classifies a sample based on the majority class among its nearest neighbors using Euclidean distance.
- predict_proba() provides the probability of a sample belonging to each class.

Tools / Software Required:

Hardware: Computer/Laptop, Minimum 4 GB RAM

Software: Python 3.x, Jupyter Notebook or Google Colab, Anaconda (optional)

Python Libraries: NumPy, Pandas, Matplotlib, Seaborn, Scikit-learn

Viva Questions:

1. What is supervised learning?
2. What is KNN and why KNN a lazy learner?
3. What happens when $K = 1$ and very large?
4. Why is feature scaling important in KNN?
5. What is overfitting and underfitting?
6. What is `predict_proba()`?
7. What is a decision boundary?